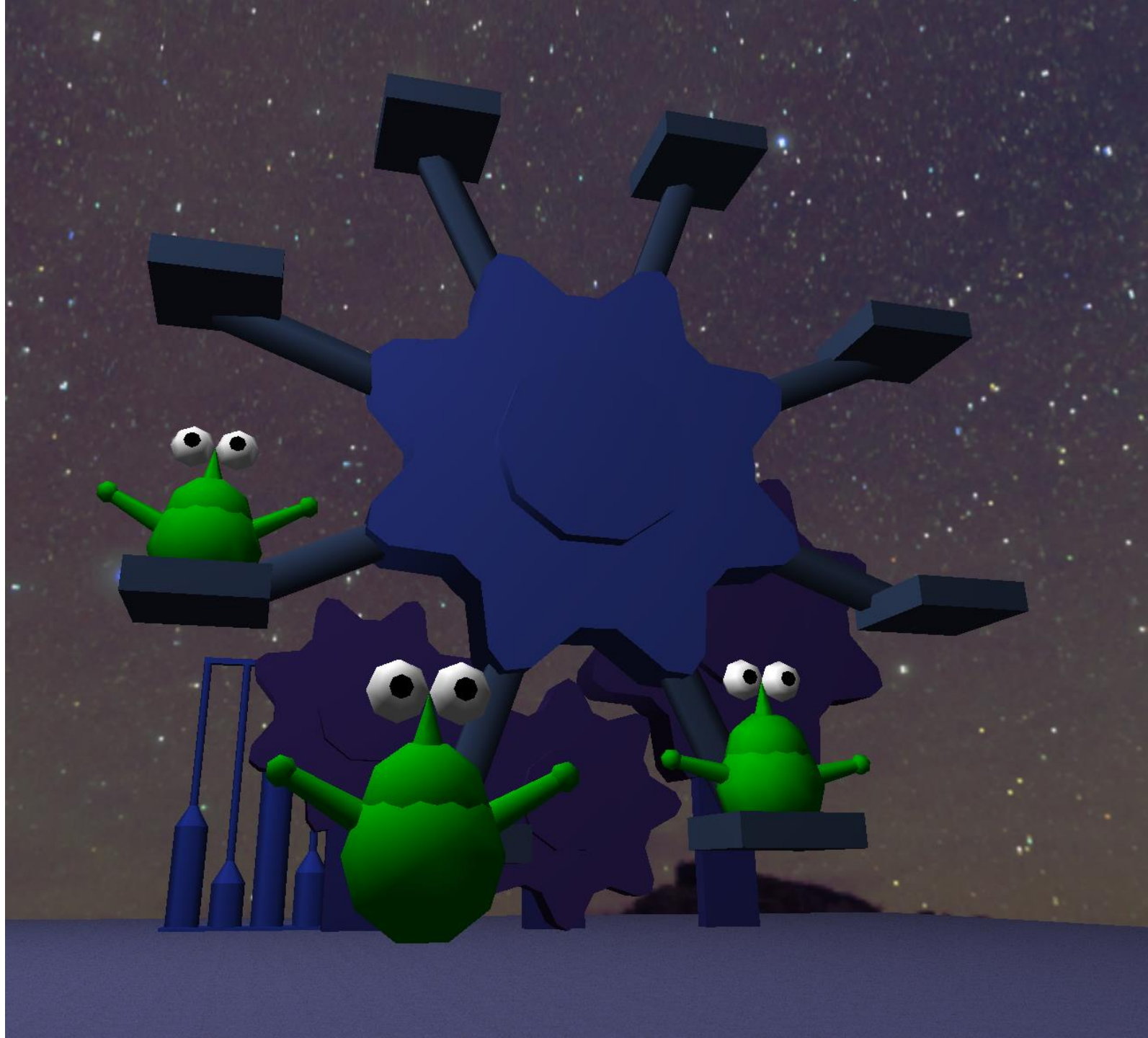
Assignment 1

Andrew Davidson



Assignment-1

- Due: 11:59pm, Thursday **26 March 2026**.
- **Late penalty**:
  - No penalty for submissions received between 27 March and 30 March (inclusive).
  - -0.5 marks for submissions received between 31 March and 3 April (inclusive).
  - -2 marks for submissions received between 4 April and 6 April (inclusive).
  - Drop-dead date and time: 11:59pm, **Monday 6 April** 2026.

- Maximum Marks: 20
- Assignment handout and this set of slides available in the ‘Assessment’ section on Learn page.

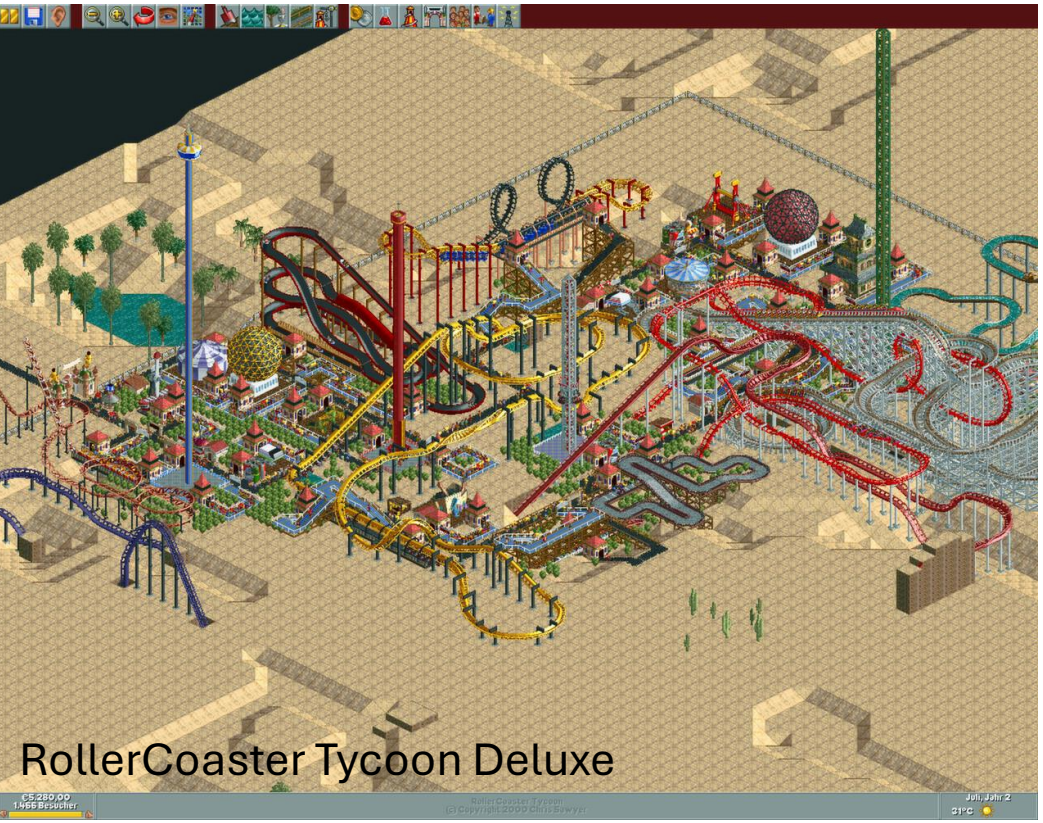
Assignment-1 Submission

- Use only C/C++ programming language and OpenGL API
- Not a group project. Your submission must represent your own individual work
- Students may discuss assignment related problems using course forum. However, code segments or any part of your assignment submission should not be posted on Learn.

Generative AI policy

- Selective Use Allowed: Generative AI tools may be used for specific purposes within this assessment.
  - Acceptable to use GenAI for:
    - Explaining the usage of computer graphics functions
    - Answering questions about debugging
    - Direct reuse of short generated code segments (1-3 lines only)
    - Grammar checking
  - Not acceptable to use GenAI for:
    - Direct reuse of large generated code segments (4+ lines)
      - You must demonstrate **your own** understanding of the concepts and ability to implement them
    - Writing your report for you
      - There are very specific requirements for the (short) report

Theme: Alien Theme Park



RollerCoaster Tycoon Deluxe

+



Eiffel 65 - Blue

?

Demo



Requirements

- **Refer to “Assignment-1 Handout” on Learn** for the most up to date, accurate, and detailed requirements
- Basic requirements (7/20 marks)
  - Animated theme park ride model(s)
  - Animated aliens
  - Working lighting, textures
  - Event handling: simple movement and wireframe toggle
- Each requirement **must be clearly visible** to get the marks

Requirements

- **Refer to "Assignment-1 Handout" on Learn** for the most up to date, accurate, and detailed requirements
- Extra features (8/20 marks)
  - Only need some for the full 8 marks (exact marks in the handout)
  - Planar shadows
  - Complicated ride path (e.g. physics-based pendulum)
  - Aliens get on and off a ride (without teleporting)
  - Sky box/dome/sphere
  - Dynamic spotlight (moving or changing intensity)
  - Smooth camera controls
  - Custom-built surfaces (e.g. surface of revolution)
  - Dynamic surfaces (e.g. bouncing trampoline mesh)
  - Particle systems (e.g. smoke, fire, sparks)

Requirements

- **Refer to "Assignment-1 Handout" on Learn** for the most up to date, accurate, and detailed requirements
- Rendering quality (3/20 marks)
  - Marks subtracted for jittery animations, improper camera movement
  - Incorrect rendering of lighting/shadows/models
- Report (2/20 marks)
  - Read the required information in the handout carefully!
  - Listing of relevant information about your scene for the markers
  - References to external sources of models/textures etc.
  - No more than 4 pages

Ideas

- Creative implementation of the requirements is encouraged!
- Many different types of theme park rides
  - https://en.wikipedia.org/wiki/List_of_amusement_rides
- Aliens do not necessarily need to be humanoid
  - Four legs? Wings?
- Does not need to be physically accurate/plausible
  - Animations and transitions should still look smooth and designed in a deliberate way

Project Timeline

Week 3: Lab 2: Transformations for modelling and animation	Ride and alien models Scene elements, animations Planar shadows
Week 4: Lab 3: Illumination, Texture mapping, Auto texturing Camera motion	Spotlights, Double buffer Camera motion, ride pathing Texture mapping, sky boxes/domes/spheres
Week 5: Lab 4: Surface of revolution, Modelling using quad strips	Wireframe mode Procedural models Dynamic geometry
Week 6: Lab 5: Particles, Assignment help	Smoke/firework/spark emitters
Assignment Submission: 26 March	

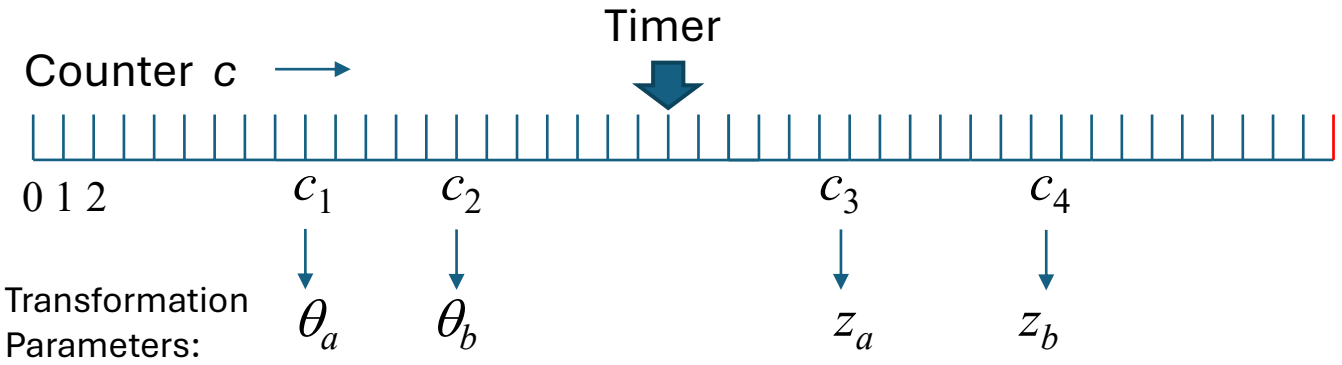
Sphere textures

- Auto texturing of spheres is covered in lab 3
- Free sphere textures available at https://polyhaven.com/hdris



Timer callback

- Please do not generate two or more independent (parallel) sequence of timer events.
- Suggested method:
  - Use only a single timer event sequence
  - Use only one timer callback function
  - Define a global `int frame` variable, incremented each time inside the timer callback function
  - Define start and end points of animation sequences based on the values of the `frame` variable.



Smooth user movement (extra criteria)

- All four movement controls (turn left and right, move forward and backward) must be changed to work smoothly
- Basic (non-smooth) camera movement is implemented in Yard.cpp from Lab-03

```
void special(int key, int x, int y) {
    if      (key == GLUT_KEY_LEFT)  angle -= 0.1;  //Change direction
    else if (key == GLUT_KEY_RIGHT) angle += 0.1;
    else if (key == GLUT_KEY_DOWN)  { //Move backward
    ...
```

  - glutSpecialFunc(special) only registers a callback for key-down events, which are not received in a smooth manner over time
  - Instead of directly incrementing the angle or position inside the callback, the current key state can be recorded
    - Instead of `angle -= 0.1`, set a global value (e.g. `turnDirection = -1`)
    - Inside your timer callback that steadily runs once per frame, change `angle` according to `turnDirection`
    - glutSpecialUpFunc(specialUp) can be used to register a callback for key-up events
      - Same parameters as glutSpecialFunc callback
      - This can be used to set `turnDirection = 0` when the user stops pressing left or right
  - glutKeyboardUpFunc can also be used to register a key-up callback for standard keys